

Full Adder Design Using Verilog

Yixu Liu 017406857

September 2, 2026

1 Objective

The objective of this experiment is to design and verify a one-bit full adder using Verilog. The Boolean logic of the full adder is derived from its truth table using Karnaugh maps (K-maps) and Sum of Products (SOP) expressions.

The full adder accepts three one-bit inputs:

- A : first input bit
- B : second input bit
- C_{in} : carry input from the previous lower-order bit

It produces two outputs:

- S : sum output
- C_{out} : carry output

2 Full Adder Operation

A full adder performs the binary addition

$$A + B + C_{in}.$$

Since the sum of three one-bit values can range from 0 to 3, a two-bit result may be required. The least significant bit becomes the sum output S , while the most significant bit becomes the carry output C_{out} .

For example,

$$1 + 1 + 0 = 2 = (10)_2.$$

Therefore,

$$C_{out} = 1$$

and

$$S = 0.$$

The carry input C_{in} represents a carry generated by the previous lower-order bit. When multiple full adders are connected together, the carry output of one full adder becomes the carry input of the next full adder.

$$C_{out}(n) \rightarrow C_{in}(n+1)$$

3 Truth Table

The complete truth table for the one-bit full adder is shown in Table 1.

A	B	C_{in}	S	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

4 Sum Output Logic Using K-Map and SOP

From the truth table, the sum output S is equal to 1 for the following input combinations:

$$001, \quad 010, \quad 100, \quad 111.$$

Therefore,

$$S = \Sigma m(1, 2, 4, 7).$$

The corresponding canonical SOP expression is

$$S = \overline{A} \overline{B} C_{in} + \overline{A} B \overline{C_{in}} + A \overline{B} \overline{C_{in}} + A B C_{in}.$$

The three-variable K-map for S is shown below.

	BC_{in}			
A	00	01	11	10
0	0	1	0	1
1	1	0	1	0

The columns are arranged in Gray-code order:

$$00, 01, 11, 10.$$

Each cell containing a 1 is surrounded by 0s. Therefore, none of the 1s can be combined into groups of two or four.

As a result, the minimal SOP expression remains

$$S = \overline{A} \overline{B} C_{in} + \overline{A} B \overline{C_{in}} + A \overline{B} \overline{C_{in}} + A B C_{in}$$

This expression represents the cases in which an odd number of inputs are equal to 1.

This pattern is exactly the behavior of the XOR operation. Therefore, the SOP expression can also be written more compactly as

$$\boxed{S = A \oplus B \oplus C_{in}}$$

For implementation in Verilog, the XOR form is simpler and easier to read.

5 Carry Output Logic Using K-Map and SOP

From the truth table, $C_{out} = 1$ for the following combinations:

$$011, \quad 101, \quad 110, \quad 111.$$

Therefore,

$$C_{out} = \Sigma m(3, 5, 6, 7).$$

The canonical SOP expression is

$$C_{out} = \overline{A}BC_{in} + A\overline{B}C_{in} + AB\overline{C}_{in} + ABC_{in}.$$

The K-map for C_{out} is

A	BC_{in}			
	00	01	11	10
0	0	0	1	0
1	0	1	1	1

Unlike the sum K-map, several adjacent 1s can be grouped.

6 Final Boolean Equations

After simplifying the logic, the full adder equations are

$$\boxed{S = A \oplus B \oplus C_{in}}$$

and

$$\boxed{C_{out} = AB + AC_{in} + BC_{in}}.$$

In logic-gate form,

- XOR gates are used to generate S .
- Three AND operations and OR operations are used to generate C_{out} .

7 Verilog Implementation

The full adder was implemented using continuous assignment statements.

```
'timescale 1ns / 1ps

module full_adder(
    input wire A, B, Cin,
    output wire Cout, S
);

    assign S = A ^ B ^ Cin;
    assign Cout = (A & B) | (A & Cin) | (B & Cin);

endmodule
```

The statement

```
assign S = A ^ B ^ Cin;
```

implements

$$S = A \oplus B \oplus C_{in}.$$

The Verilog operator `^` represents XOR.

The statement

```
assign Cout = (A & B) | (A & Cin) | (B & Cin);
```

implements

$$C_{out} = AB + AC_{in} + BC_{in}.$$

The operator `&` represents AND, while `|` represents OR.

Because the full adder is a combinational logic circuit, continuous assignment statements are appropriate. No clock signal or storage element is required.

8 Testbench

A Verilog testbench was created to verify the operation of the full adder.

The testbench uses `reg` variables for the three inputs:

```
reg A;
reg B;
reg Cin;
```

The outputs are declared as wires:

```
wire S;
wire Cout;
```

The full adder module is instantiated as the unit under test (UUT):

```
full_adder uut (
    .A(A),
    .B(B),
    .Cin(Cin),
    .S(S),
    .Cout(Cout)
);
```

The complete testbench used in this experiment is shown below.

```
'timescale 1ns / 1ps

module full_adder_tb;

    reg A;
    reg B;
    reg Cin;

    wire S;
    wire Cout;

    full_adder uut (
        .A(A),
        .B(B),
        .Cin(Cin),
        .S(S),
        .Cout(Cout)
    );

    initial begin
        A = 0; B = 0; Cin = 0;
        #10;

        A = 0; B = 0; Cin = 1;
        #10;

        A = 0; B = 1; Cin = 0;
        #10;

        A = 0; B = 1; Cin = 1;
        #10;

        A = 1; B = 0; Cin = 0;
        #10;

        A = 1; B = 0; Cin = 1;
        #10;
```

```

        A = 1; B = 1; Cin = 0;
        #10;

        A = 1; B = 1; Cin = 1;
        #10;

        $finish;
    end

endmodule

```

The testbench applies every possible input combination of A , B , and C_{in} . Since there are three one-bit inputs, the total number of possible combinations is

$$2^3 = 8.$$

Each input combination is applied for 10 ns. The expected simulation results are shown below.

Time (ns)	A	B	C_{in}	S	C_{out}
0–10	0	0	0	0	0
10–20	0	0	1	1	0
20–30	0	1	0	1	0
30–40	0	1	1	0	1
40–50	1	0	0	1	0
50–60	1	0	1	0	1
60–70	1	1	0	0	1
70–80	1	1	1	1	1

At 80 ns, the `$finish` command terminates the simulation.

Testing all eight input combinations verifies that the implemented full adder follows the expected truth table.

9 Conclusion

A one-bit full adder was designed and implemented using Verilog. The logic equations were derived from the truth table using Karnaugh maps and SOP expressions.

For the sum output,

$$S = \Sigma m(1, 2, 4, 7),$$

which gives the SOP expression

$$S = \overline{A}BC_{in} + \overline{A}B\overline{C}_{in} + A\overline{B}C_{in} + ABC_{in}.$$

The K-map shows that the four minterms cannot be grouped further. This logic corresponds to the XOR relationship

$$S = A \oplus B \oplus C_{in}.$$

For the carry output,

$$C_{out} = \Sigma m(3, 5, 6, 7),$$

and K-map simplification produces

$$C_{out} = AB + AC_{in} + BC_{in}.$$

The design was then verified using a Verilog testbench that tested all eight possible input combinations. The expected outputs match the full-adder truth table, demonstrating the correct operation of the circuit.